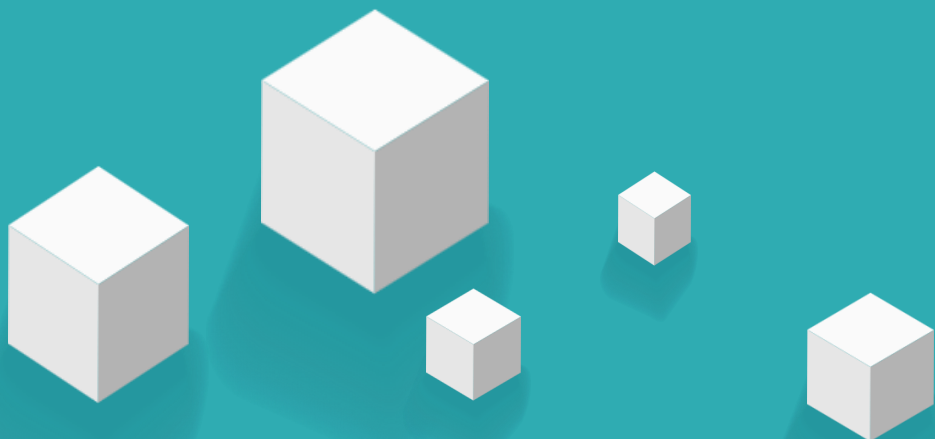
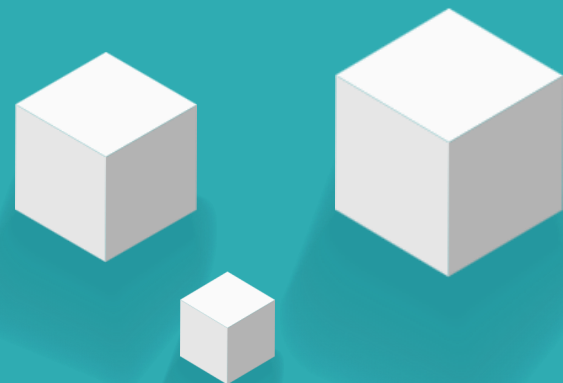


vue.js
Aug 12

第五章 结构渲染



第5章 结构渲染

- 条件渲染指令
- 列表渲染指令

5.1 条件渲染指令

在Vue中，当需要根据不同的判断结果显示不同的DOM元素时，可以通过条件渲染指令来实现。条件渲染指令可以辅助开发者按需控制DOM元素的显示与隐藏。

条件渲染指令：

- v-if
- v-show

5.1 条件渲染指令

1. v-if

v-if是根据布尔值切换元素的显示或隐藏状态，本质是通过操作DOM元素来切换显示状态。

- 当给定的值为**true**时，元素存在于DOM树中；
- 当给定的值为**false**时，元素从DOM树中移除。

5.1 条件渲染指令

v-if有两种使用方式。



直接给定一个条件，控制单个元素的显示或隐藏

```
<标签名 v-if="条件">  
</标签名>
```

通过v-if结合v-else-if、v-else来控制不同元素的显示或隐藏

```
<标签名 v-if="条件A">展示A</标签名>  
<标签名 v-else-if="条件B">展示B</标签名>  
<标签名 v-else>展示C</标签名>
```

5.1 条件渲染指令

v-if的使用方法

```
<div id="root">
```

小明的学习评定等级为：

```
<p v-if="type === 'A'">优秀</p>
```

```
<p v-else-if="type === 'B'">良好</p>
```

```
<p v-else>差</p>
```

```
<button @click="type = 'A'">切换成优秀</button>
```

```
<button @click="type = 'B'">切换成良好</button>
```

```
<button @click="type = 'C'">切换成差</button>
```

```
</div>
```

```
<script >
```

```
  new Vue({
```

```
    el:'#root',
```

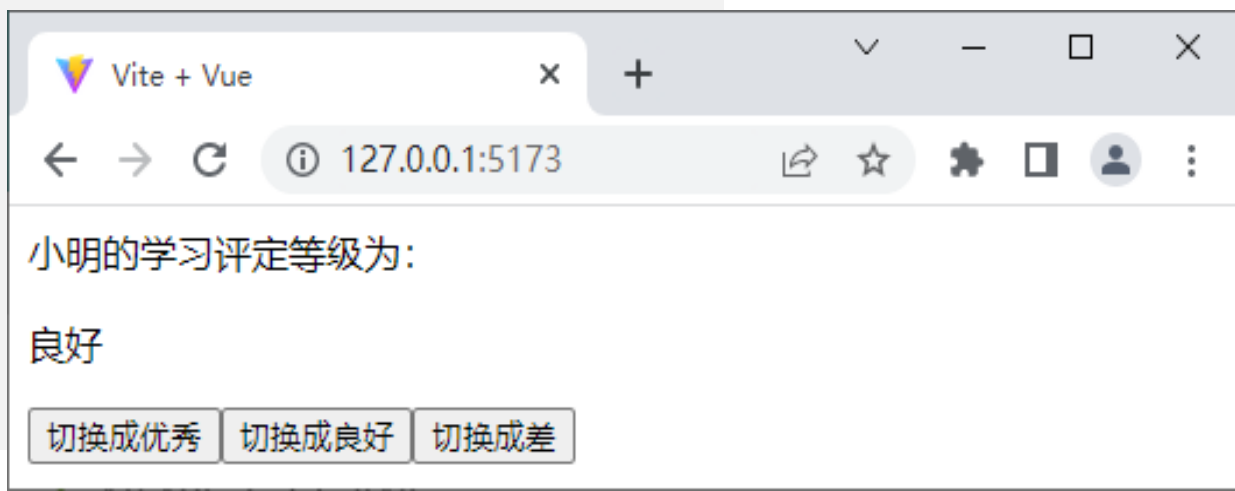
```
    data:{
```

```
      type:'B'
```

```
    }
```

```
  })
```

```
</script>
```



5.1 条件渲染指令

2. v-show

- `v-show`与`v-if`都用来决定某一个元素是否在页面上显示出来。
- `v-show`的原理是通过为元素添加或移除`display: none`样式来实现元素的显示或隐藏。
- 比较

v-if	v-show
■ 适用切换频率较低的场景 ■ 不展示的DOM元素直接移除 ■ 可以和<code>v-else-if</code>、<code>v-else</code>一起用，但不能被打断	■ 适用切换频率较高的场景 ■ 不展示的DOM元素未被移除，只是利用样式隐藏

5.1 条件渲染指令

v-show的使用方法

```
<div>
  <p v-if="flag">通过v-if控制的元素</p>
  <p v-show="flag">通过v-show控制的元素</p>
  <button @click="flag = !flag">显示/隐藏
</button>
</div>
<script >
  new Vue({
    el:'#root',
    data:{
      flag:true
    }
  })
</script>
```



5.2 列表渲染指令

- 开发购物应用时，为了方便用户查找商品信息，通常会以商品列表的形式展示商品信息。在商品列表中，每件商品的结构都是相同的，如果每件商品的结构都要手动定义，会非常麻烦。
- Vue提供了列表渲染指令v-for。开发者只需在模板中定义一件商品的结构，v-for便会根据开发者提供的数据自动渲染商品列表中所有的商品。

5.2 列表渲染指令

使用v-for循环渲染列表

根据数组渲染列表 >

<标签名 v-for="(item, index) in arr"> </标签名>

根据对象渲染列表 >

<标签名 v-for="(item, name, index) in obj">
</标签名>

根据数字渲染列表 >

<标签名 v-for="(item, index) in num"> </标签名>

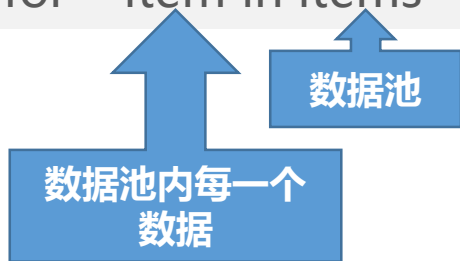
根据字符串渲染列表 >

<标签名 v-for="(item, index) in str"> </标签名>

5.2 列表渲染指令

- 使用v-for（根据list数组中的元素）渲染列表后，当在list数组中删除一个元素后，index会发生变化，v-for会重新渲染列表，导致性能下降。
- 为了给v-for一个提示，以便它能跟踪每个节点的身份，从而对现有元素进行重用和重新排序，建议通过key属性为列表中的每一项提供具有唯一性的值，示例代码如下。

```
<div v-for="item in items" :key="item.id"> </div>
```

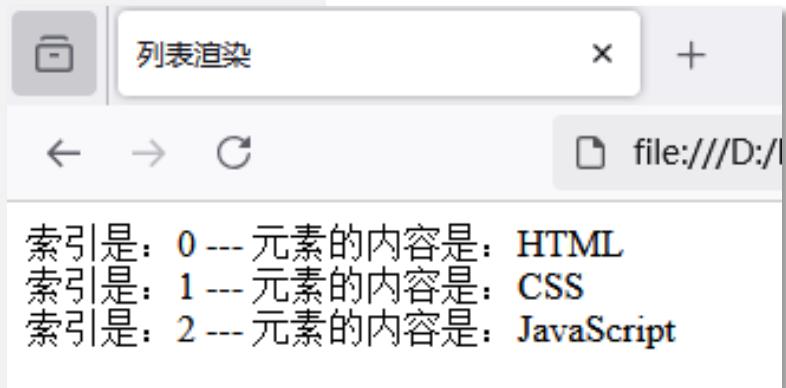


5.2 列表渲染指令

1、使用v-for根据一维数组渲染列表

遍历时的索引值

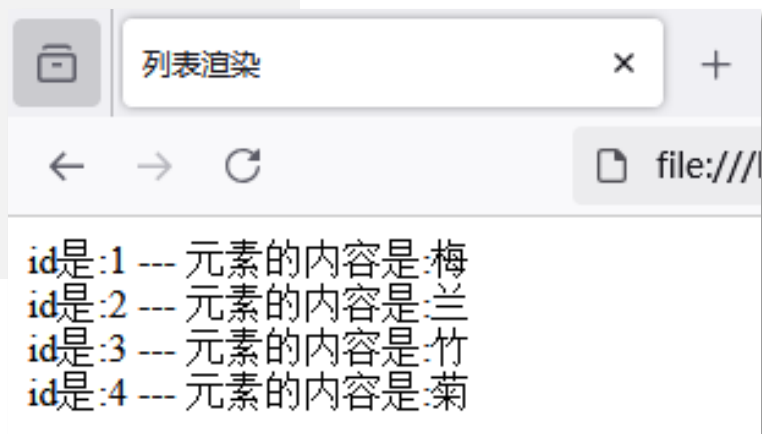
```
<div id="root">
  <div v-for="(item, index) in list" :key="index">
    索引是：{{ index }} --- 元素的内容是：{{ item }}
  </div>
</div>
<script>
  new Vue({
    el:'#root',
    data:{
      list: ['HTML', 'CSS', 'JavaScript']
    }
  })
</script>
```



5.2 列表渲染指令

2、使用v-for根据对象数组渲染列表

```
<div id="root">
  <div v-for="item in list" :key="item.id">
    id是:{{ item.id }} --- 元素的内容是:{{ item.message }}
  </div>
</div>
<script >
  new Vue({
    el:'#root',
    data:{
      list: [
        { id: 1, message: '梅', }, { id: 2, message: '兰', },
        { id: 3, message: '竹', }, { id: 4, message: '菊', }
      ]
    }
  })
</script>
```

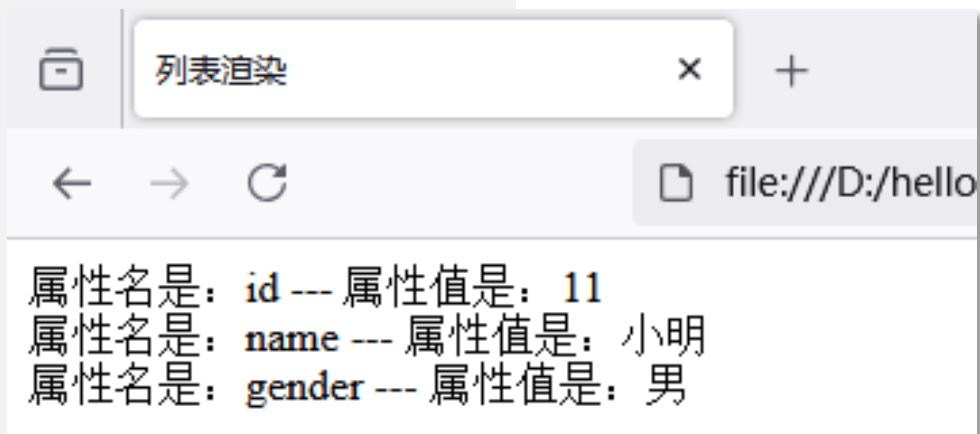


5.2 列表渲染指令

3、使用v-for根据对象渲染列表

```
<div id="root">  
  <div v-for="(value, name) in user" :key="name">  
    属性名是：{{ name }} --- 属性值是：{{ value }}  
  </div>  
</div>
```

```
<script >  
  new Vue({  
    el:'#root',  
    data:{  
      user: {  
        id: 11, name: '小明', gender: '男'  
      }  
    }  
  })  
</script>
```



当遍历 **数组** 时语法是 (item, index)
当遍历 **对象** 时语法是 (value, name, index)

练习1参考代码

```
<script>
  new Vue({
    el:'#root',
    data:{
      City:"",
      Name:"",
      n:0,
      List:[]
    },
    computed: {
      fullName(){
        return `${this.City} 的 ${this.Name} 赞了你`
      }
    },
    methods: {
      Combine() {
        this.List.push(this.fullName)
        this.n+=1
      }
    }
  })
</script>
```

```
<div id="root">
  <h3>此刻的心情</h3>
   <br/>
  城市： <input type="text" v-model='City'> <br/>
  姓名： <input type="text" v-model='Name'> <br/>
  <button @click="Combine">赞{{n}}</button> <br/> <br/>
  <p v-for="(item, index) in List" :key="index">
    {{ item }}
  </p>
</div>
</body>
```